

Laboratoire 02 – Programme simple avec E/S de la DE1-SoC

Objectifs du laboratoire

Ce laboratoire a pour but d'accéder à des entrées et sorties de la carte DE1-SoC. Vous devrez utiliser le plan d'adressage fournit pour configurer et commander les périphériques PIO déjà instanciés dans le projet, et ainsi utiliser les E/S. Vous écrirez un programme simple pour piloter les E/S selon les spécifications données ci-après.

Spécifications

Plan d'adressage

Voici le plan d'adressage qui est utilisé dans le projet fournit.

Offset on bus AXI lightweight HPS-to-FPGA (relative to BA_LW_AXI)	Fonctionnalités
0x00_0000 – 0x00_0003	Constante ID 32 bits (Read only)
0x00_0004 - 0x00_00FF	reserved
0x00_0100 - 0x00_011F	Périphérique PIO 0 pour interrupteurs(switch), leds et boutons (KEY).
0x00_0120 - 0x00_013F	Périphérique PIO 1 pour les 4 afficheurs 7 segments (HEX0, HEX1, HEX2 et HEX3).
0x00_0140 – 0x1F_FFFF	not used

Configuration utilisée dans le périphérique PIO 0

PIO Bits	31	26	25	16	15 - 14	13	4	3	0
E/S connectés	unused		Leds (9...0)		unused	Switch (9...0)		Key (3...0)	

- Les ports sont configurés en mode "Bidirectionnel".
- "Individual bit set/clear output register" → activé.
- "Edge capture register" → désactivé.
- "Generate IRQ" → désactivé.
- Les switches et Leds sont actifs à l'état haut. Les keys sont actifs à l'état bas.

Configuration utilisée dans le périphérique PIO 1

PIO Bits	31	28	27	21	20	14	13	7	6	0
E/S connectés	unused		Hex3 (6...0)		Hex2 (6...0)		Hex1 (6...0)		Hex0 (6...0)	

- Les ports sont configurés en mode "Bidirectionnel"
- "Individual bit set/clear output register" → activé.
- "Edge capture register" → désactivé.
- "Generate IRQ" → désactivé.
- Les afficheurs 7 segments sont actifs à l'état bas.

Programme

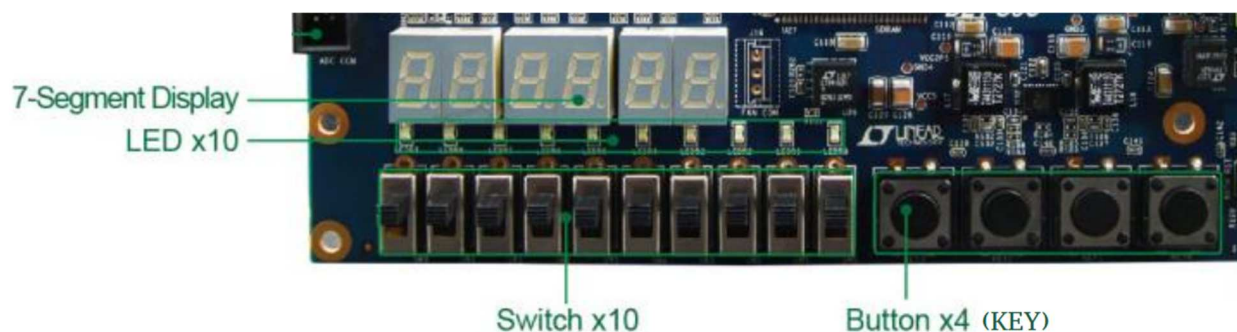
Le but est d'allumer les LEDs et les afficheurs 7 segments selon l'état des boutons (KEY) et interrupteurs (switch - SW) disponibles. La spécification du fonctionnement est la suivante :

Au démarrage, le programme doit remplir les conditions suivantes :

- Les leds affichent le paterne "0101010101".
- Les afficheurs 7 segments Hex0 et Hex 1 sont éteints.
- L'afficheur 7 segments Hex2 affiche la valeur 0.
- L'afficheur 7 segments Hex3 a le segment du bas allumé.
- Afficher la constante ID au format hexadécimal dans la console de ARM-DS.

Ensuite pendant l'exécution du programme, à tout instant les actions suivantes doivent être respectées :

- Pression sur KEY0, selon état SW9-8 :
 - SW9-8 = 00 : Copie de la valeur de SW7-0 sur les Leds 7-0.
 - SW9-8 = 01 : Set des bits actifs de SW7-0 sur les Leds 7-0.
 - SW9-8 = 10 : Clear des bits actifs de SW7-0 sur les Leds 7-0.
 - SW9-8 = 11 : Toggle de la valeur actuelle des Leds 7-0.
- Pression sur KEY1 :
 - La valeur de SW7-0 interprétée en hexadécimal est affichée sur les afficheurs 7 segments Hex0 (SW3 à 0) et Hex1(SW7 à 4).
- Pression sur KEY2 :
 - Incréments de 1 la valeur hexadécimale de l'afficheur 7 segments Hex2.
- Pression sur KEY3 :
 - Faire tourner dans le sens horaire d'une position le segment externe (un des 6 segments) de l'afficheur 7 segments Hex3.



Travail demandé

- 1) Rechercher dans la documentation du HPS (*cyclone-v_hps_trm_5v4.pdf*) la base adresse du bus AXI lightweight HPS-to-FPGA.
- 2) Sur Cyberlearn, récupérer l'archive du projet et la décompresser. Puis générer le bitstream du projet.
- 3) Créer un nouveau projet ARM Development Studio et placer le workspace dans « soft/proj ».
- 4) Vérifier la lecture de la constante ID depuis l'onglet memory (mode debug).
- 5) Lire les informations sur le PIO core dans la documentation "Embedded Peripherals IP User Guide" (*ug_embedded_ip_2018.pdf*). Identifier les registres du PIO core et les fonctionnalités associées.
- 6) Depuis l'onglet memory dans ARM-DS, vérifier l'utilisation des registres du PIO identifié précédemment. Pour cela, allumer des leds et lire l'état des switches.
- 7) Compléter le fichier header « axi_lw.h » dans le dossier « soft/src » avec la base adresse du bus AXI lightweight HPS-to-FPGA. Vous pourrez ensuite utiliser dans votre code la macros "AXI_LW_REG(offset_value)" pour faire des accès en écriture et lecture sur ce bus, avec en paramètre la valeur de l'offset choisi.
- 8) Compléter les "#define ..." du fichier header « pio_function.h » dans le dossier « soft/src ». Puis écrire les fonctions d'initialisations dans le fichier « pio_function.c » en respectant les prototypes donnés dans le fichier header : Switchs_init(), Leds_init(), Keys_init(), Segs7_init().
- 9) Tester vos fonctions d'initialisations sur la carte DE1-SoC depuis le programme « hps_application.c ».
- 10) Ecrire les fonctions de lecture et d'écriture des entrées sorties dans le fichier « pio_function.c » en respectant les prototypes donnés dans le fichier header : Switchs_read, Leds_write, Leds_set, Leds_clear, Leds_toggle, Key_read, Seg7_write, Seg7_write_hex.
- 11) Tester vos fonctions sur la carte DE1-SoC depuis le programme « hps_application.c ».
- 12) Ecrire le programme « hps_application.c » à l'aide des fonctions écrites précédemment. Le programme doit réaliser les fonctionnalités définies dans les spécifications.

13) Tester votre programme sur la carte DE1-SoC.

14) Faire valider votre programme par le professeur ou l'assistant.

À rendre

Ce laboratoire est évalué par une évaluation orale. Il n'y a pas de rapport à rédiger. Un contrôle du travail sera effectué. Vous devez rendre une archive avec les sources du projet pour Quartus et le programme C. Utiliser le Makefile à la racine du projet pour générer votre archive à rendre en tapant « make zip » dans un terminal.

Les fichiers sont à rendre sur Cyberlearn à la date indiquée.